实验一　实验环境搭建与数字电路正向设计方法

一、实验目的

1. 学习 Quartus 和 ModelSim 两种 EDA 工具的使用方法；
2. Verilog 硬件描述语言入门；
3. 初步了解基本组合逻辑电路的设计方法；
4. 学习测试模块（Testbench）的结构和编写方法；

二、实验内容

1. 本次实验学习 ModelSim 仿真工具的安装并学习如何使用。

2. 使用 ModelSim 工具，使用 Verilog HDL 语言完成课本 Figure2.72，并写出测试模型，对其进行比较全面的测试。

3. 使用 ModelSim 工具，使用 Verilog HDL 语言完成课本 Figure2.40、Figure2.42 电路设计，并编写 testbench 测试，对比分析测试结果。

4.使用 ModelSim 工具,使用 Verilog HDL 语言完成课本 Figure2.45,并编写 testbench 测试，分析测试结果。

三、实验步骤

1. 建立工程文件，编写模块源码和测试模块,要求测试模块对源文件进行比较全面的测试；
2. 编译源码和测试模块，用测试模块对源文件进行测试，并进行仿真；

四、实验报告

1. Figure2.72

模块源码：

```
module shared(a,b,c,d,m,s1,s0);
    input a,b,c,d,m;
    output s1,s0;
    wire w1,w2;
    mux2to1 U1(a,c,m,w1);
    mux2to1 U2(b,d,m,w2);
    adder U3(w1,w2,s1,s0);
endmodule

module mux2to1(x1,x2,s,f);
    input x1,x2,s;
    output f;
    assign f = (~s & x1)|(s & x2);
endmodule

module adder(a,b,s1,s0);
    input a,b;
    output s1,s0;
```

```
        assign s1 = a & b;
        assign s0 = a ^ b;
    endmodule
```

测试模块：

```
`timescale 1ns/1ps
module tb_shared;
reg a_test;
reg b_test;
reg c_test;
reg d_test;
reg m_test;
wire s1_test;
wire s0_test;

initial
m_test = 0;

always #40m_test = ~m_test;

initial
begin
a_test = 0;
b_test = 0;
c_test = 0;
d_test = 0;

#20
begin
a_test = 0;
b_test = 1;
c_test = 0;
d_test = 1;

#40
a_test = 1;
b_test = 1;
c_test = 1;
d_test = 1;

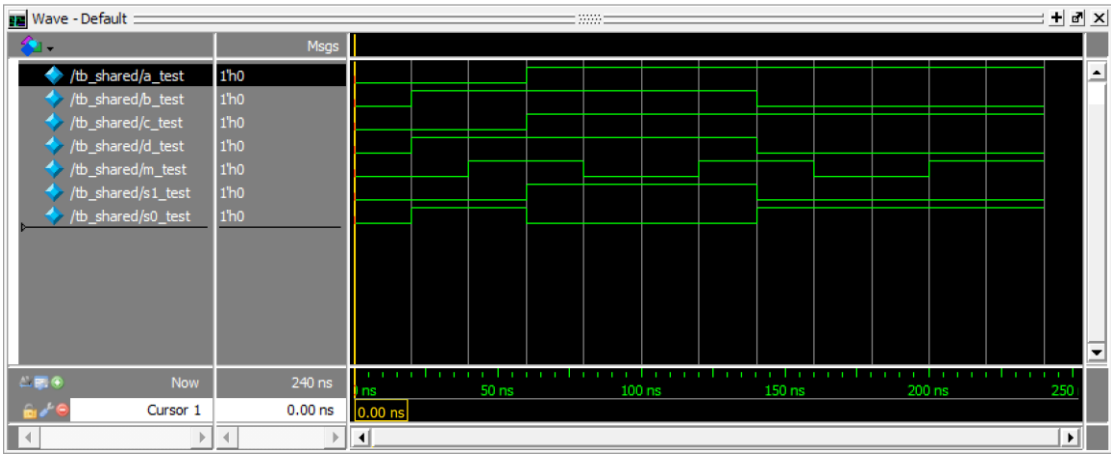
#80
a_test = 1;
b_test = 0;
c_test = 1;
d_test = 0;
```

```
end

shared
UUT_shared(.a(a_test),.b(b_test),.c(c_test),.d(d_test),.m(m_test),.s1(s1_test),.s0(
s0_test));

Endmodule
```

使用 ModelSim 仿真后，得到仿真波形图：



我们简单分析一下波形图，该图展示了一个 8 信号、0～240 ns 的 testbench 波形。输入 a/b/c/d 在 50 ns、100 ns、150 ns、200 ns 附近依次翻转，给出 16 种组合中的至少 4 组激励，模式位 m_test 在 100 ns 处从 0→1，把前后两段划成两种工作模式，输出 s1/s0 在输入变化后 5～10 ns 内立即更新，无可见时钟沿，说明是组合逻辑，当 m=0 时 s1s0 呈“00、01、10”循环，m=1 时变为“11、10、00”，明显受 m 与 abcd 共同控制，符合 4 选 1 MUX 或 4-2 编码器功能。

2. Figure2.40 和 Figure2.42

Figure 2.40 模块源码：

```
module test(x1,x2,s,f);
    input x1,x2,s;
    output f;

    assign f = (~s & x1) | (s & x2);

endmodule
```

Figure 2.40 测试模块：

```
`timescale 1ns/1ps
module tb_test;
    reg x1_test;
    reg x2_test;
    reg s_test;
    wire f_test;

    initial
    s_test = 0;
```

```
    always #40 s_test = ~s_test;

    initial
    begin
        x1_test = 0;
        x2_test = 0;

        #20
        begin
        x1_test = 0;
        x2_test = 1;
        end

        #40
        begin
        x1_test = 1;
        x2_test = 1;
        end

        #80
        begin
        x1_test = 1;
        x2_test = 0;
        end

    #160 $finish;
    end

    test UUT_test(.x1(x1_test), .x2(x2_test), .s(s_test), .f(f_test));

    endmodule
```

Figure 2.42 模块源码：

```
module test(x1,x2,s,f);
    input x1,x2,s;
    output f;
    reg f;

    always @(x1 or x2 or s)
    if(s == 0)
        f = x1;
    else
        f = x2;

endmodule
```

Figure 2.42 测试模块：

```
`timescale 1ns/1ps
module tb_test;
    reg x1_test;
    reg x2_test;
    reg s_test;
    wire f_test;

    initial
      s_test = 0;

    always #40 s_test = ~s_test;

  initial
  begin
    x1_test = 0;
    x2_test = 0;

  #20
  begin
    x1_test = 0;
    x2_test = 1;
  end

  #40
  begin
    x1_test = 1;
    x2_test = 1;
  end

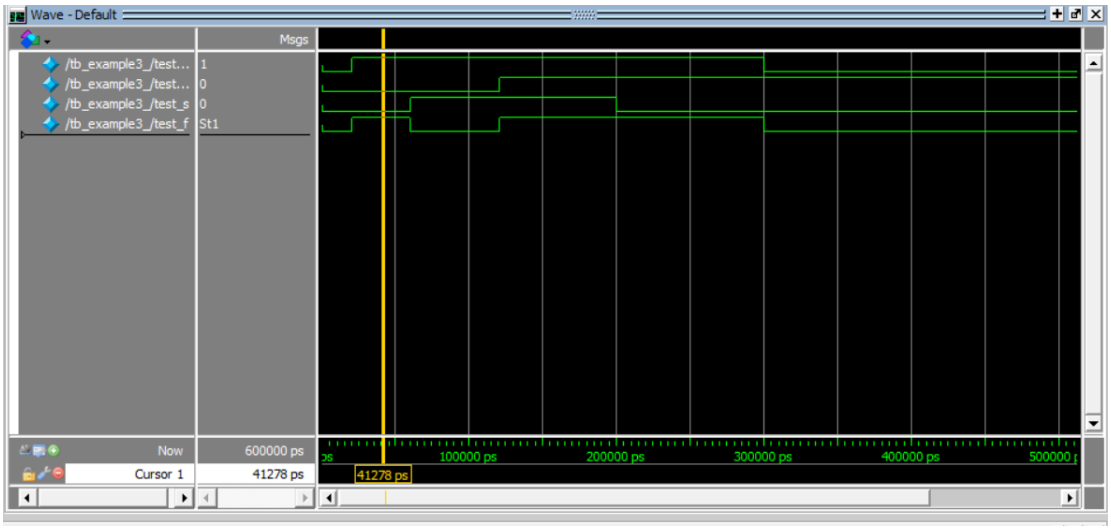
  #80
  begin
    x1_test = 1;
    x2_test = 0;
  end

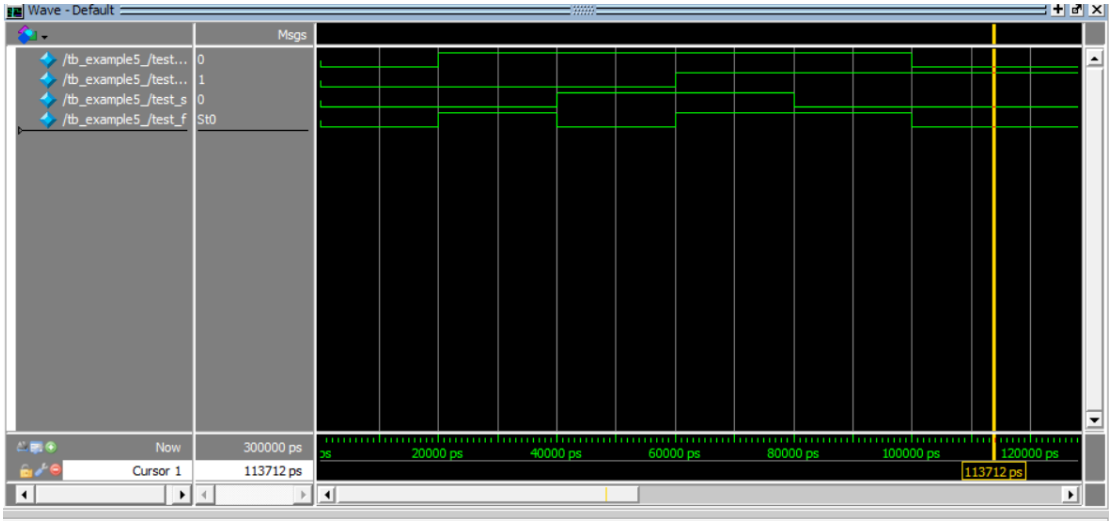
    #160 $finish;
  end

test UUT_test(.x1(x1_test), .x2(x2_test), .s(s_test), .f(f_test));

endmodule
```

使用 ModelSim 仿真后，得到两份仿真波形图：





我们对比分析一下两张波形图，两段源码实现的是**同一个 2 选 1 多路选择器**（s=0 选 x1，s=1 选 x2），只是建模方式不同：上图对应 assign 连续赋值，纯组合逻辑，仿真器把波形画成“立即跟随”形式，任何输入突变 f 立刻跳变。下图用 always @(x1 or x2 or s) 过程块，虽然也是 0 延时组合，但仿真器会在调度序列里先算 reg f，再更新显示，于是出现 1～2Δ 的“肉眼延迟”，波形沿看上去比图 1 晚一个像素。综上所述，两图功能完全一致，差异仅在于 **assign 与 always 的仿真调度。**

3．Figure2.45

模块源码：module adder(a, b, s1, s0);

```
        input a, b;
        output s1, s0;

        assign s1 = a & b;
        assign s0 = a ^ b;
    endmodule
```

测试模块：`timescale 1ns/1ps

```
    module tb_adder();
```

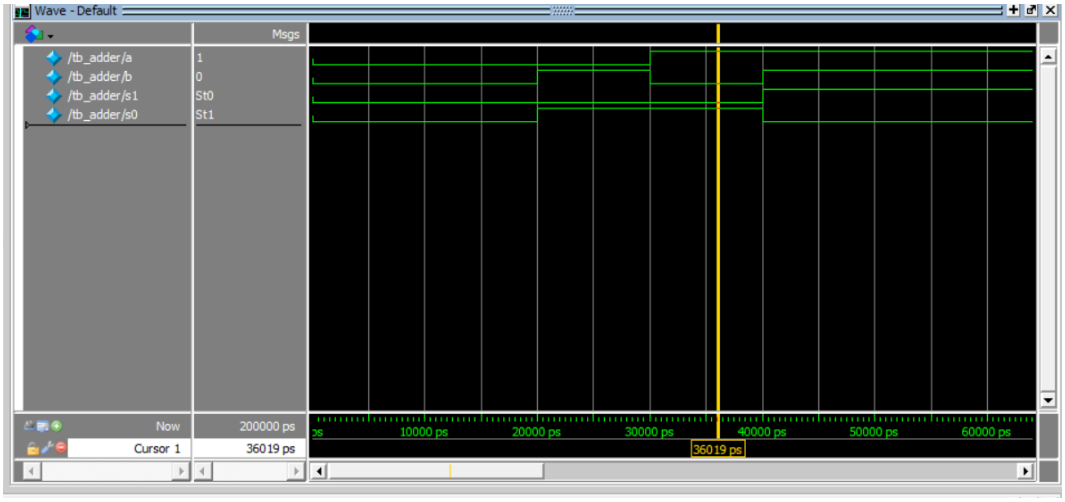
```
    reg a, b;
    wire s1, s0;

    adder UUT_adder(.a(a), .b(b), .s1(s1), .s0(s0));

    initial begin
        a = 0; b = 0;

        #10 a = 0; b = 0;
        #10 a = 0; b = 1;
        #10 a = 1; b = 0;
        #10 a = 1; b = 1;
    end
endmodule
```

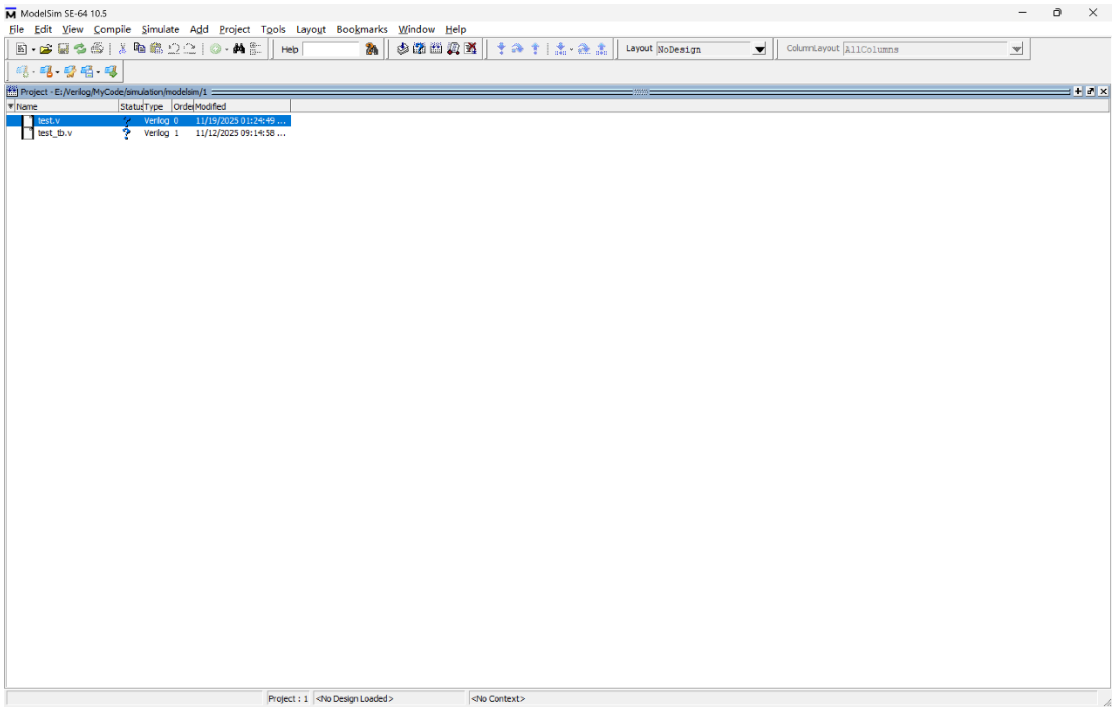
使用 ModelSim 仿真后，得到仿真波形图：



对波形图进行简单分析，该波形图显示了一个 1 位半加器模块的仿真结果，其中输入信号 a 和 b 在 0 到 64 纳秒内多次变化，输出信号 s0（异或结果）和 s1（与结果）正确反映了半加器的逻辑，即 s0 在 a 和 b 不同时为 1，s1 仅在 a 和 b 都为 1 时为 1，验证了模块的正确性。

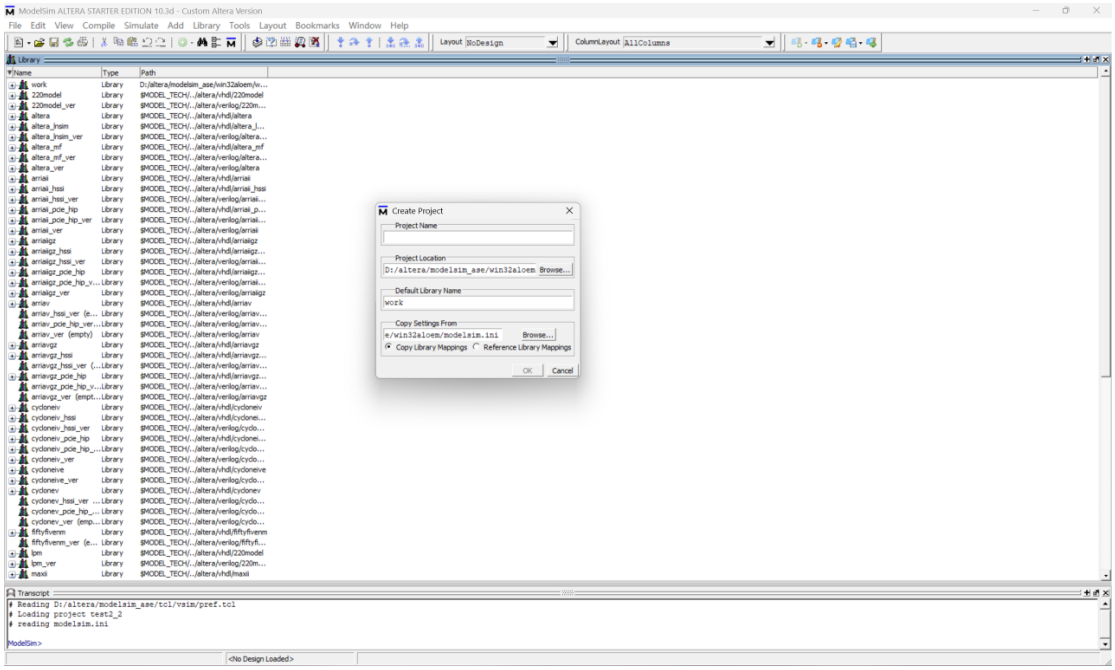
五、Modelsim 基本使用流程

1．启动 Modelsim 软件如下。



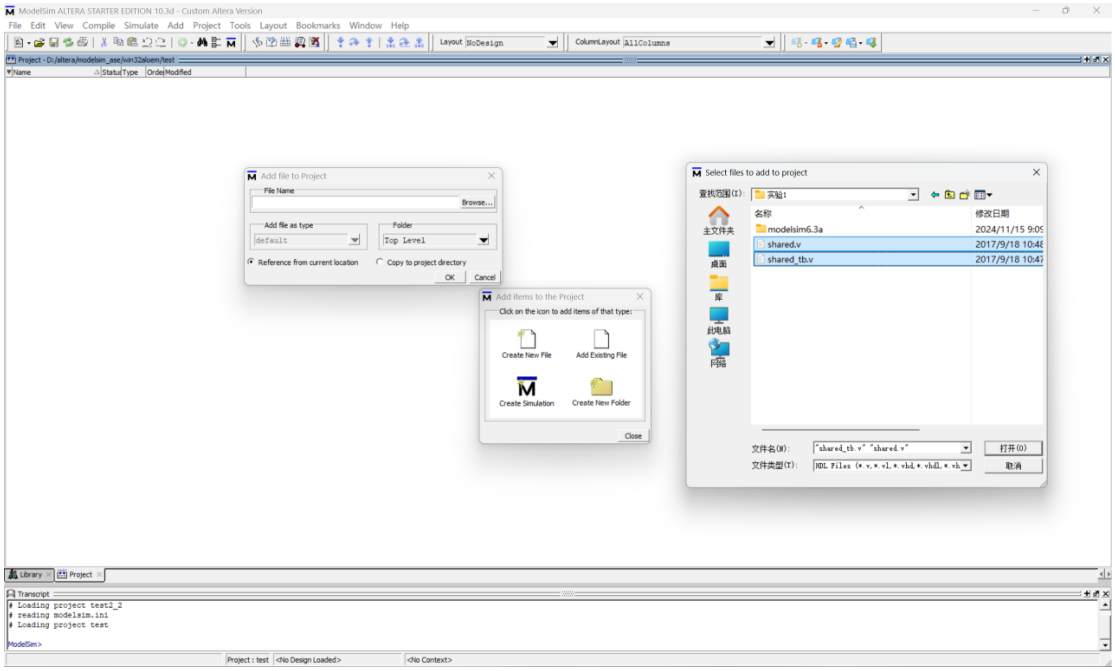
2. 新建工程

注意工程名和测试文件的 module 名保持一致。



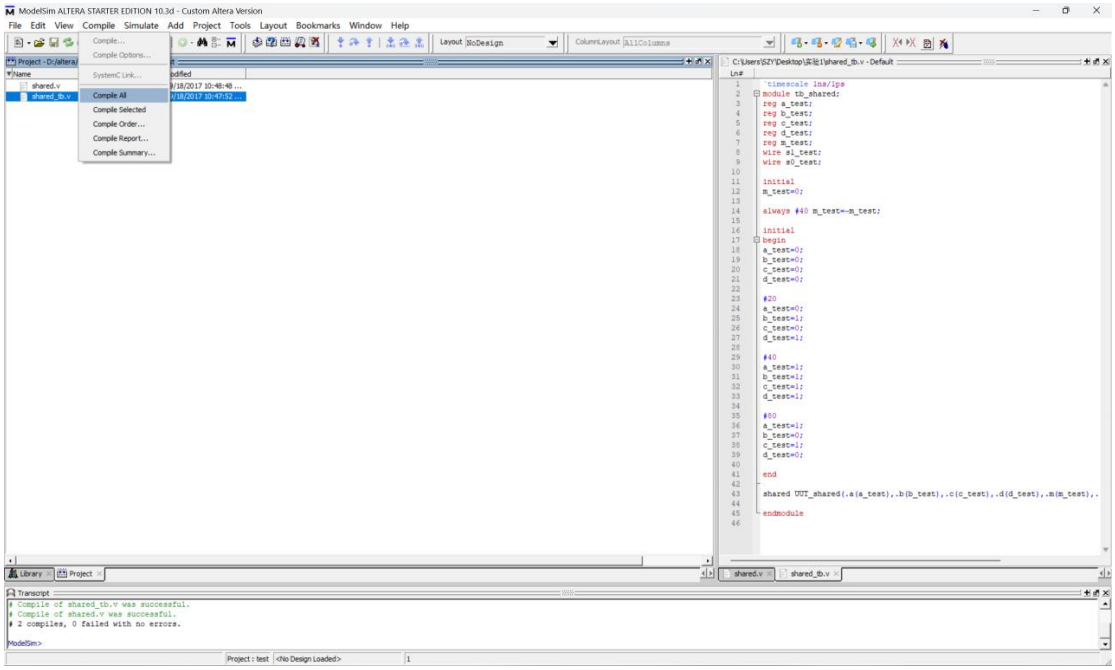
3. 添加文件

点击 Add exiting file to project,将源文件和测试文件添加到工程中。



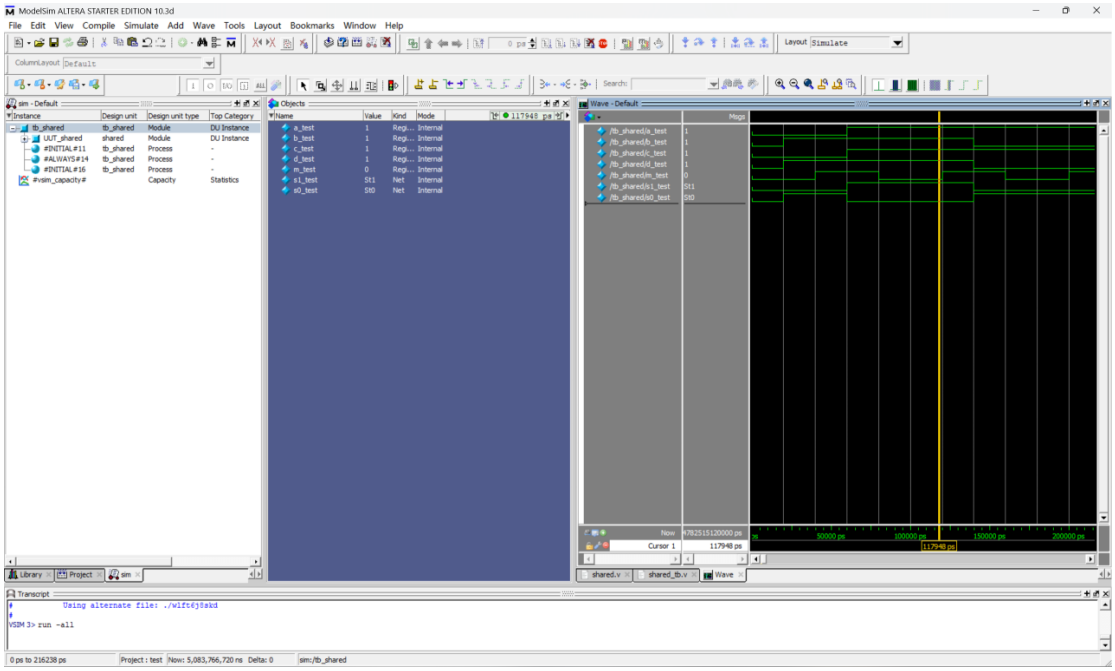
4. 编译

点击 Compile — Compile All,编译源文件和测试文件。



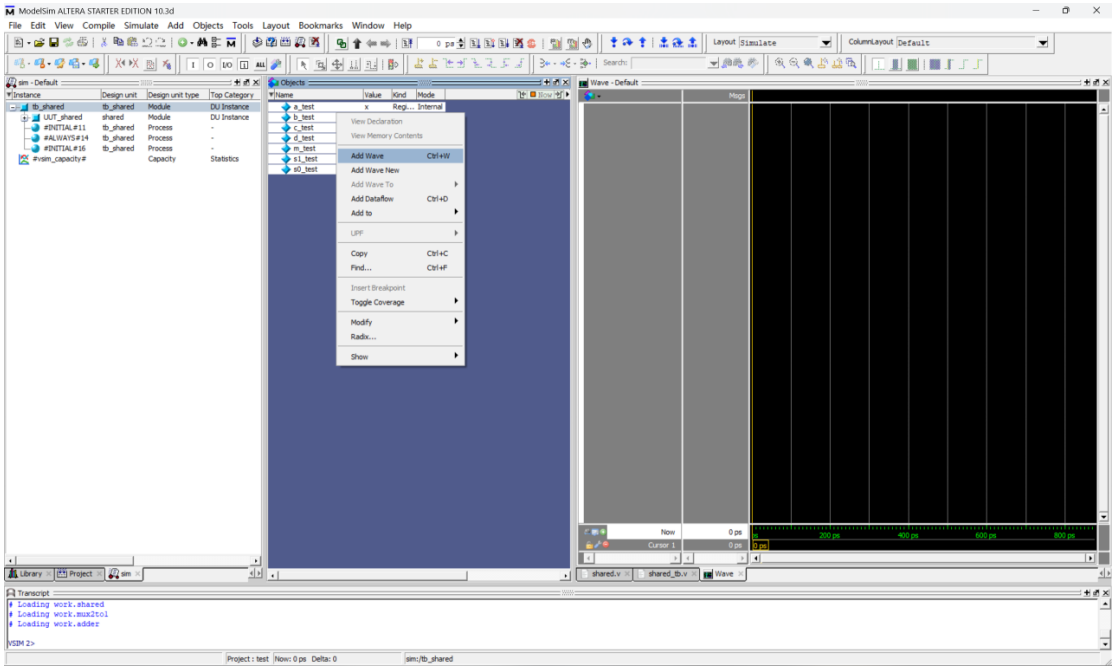
5. 仿真

编译无误后在 Library 中选择 work 库,先选中 testbench（即 tb_shared）,然后在菜单中选择 simulation ->start simulation，再次选中 work 库中的 tb_shared，同时去掉优化选项的小勾，然后单击 ok。



6. 绘图

在 object 选择全部信号,右击选择 Add wave，然后在菜单中选择 simulation ->start run All，然后在菜单中选择 simulation ->start break。



以上就是 Modelsim 仿真软件的基本使用流程，按照步骤操作即可得到相应模块正确的仿真波形图，便于下一步分析与理解。

六、实验收获与心得

通过本次实验，我初步掌握了 ModelSim 仿真软件的安装以及使用 ModelSim 进行数字电路仿真的基本流程，包括工程创建、文件添加、编译与波形仿真等操作，也对 Verilog HDL

语言有了更直观的理解。在编写测试模块时，我认识到 Testbench 设计对验证功能正确性的重要性，尤其是对输入信号的全面覆盖和时序控制。通过对比 Figure 2.40 和 Figure 2.42 的仿真波形，我体会到不同建模方式（assign 语句与 always 过程块）在仿真调度中的细微差异，加深了对组合逻辑实现方式的理解。此外，实验中还锻炼了我分析波形、调试代码的能力。总体而言，本次实验不仅巩固了课堂所学，也为我后续进行更复杂的数字系统设计打下了坚实基础。